

분산 하드코딩 전투 로직 → 데이터 주도 단일 AbilitySystem

보스·플레이어·일반몹이 제각각 하드코딩하던 공격/스킬을 하나의 파이프라인으로 통합 — 새 공격은 코드가 아니라 데이터(Effect 조합)로 만든다.

이전(분산·하드코딩) → 현재(통합 AbilitySystem) : 6가지 축의 변화

축	이전 (분산·하드코딩)	현재 (통합 AbilitySystem)
공격 정의	각 컨트롤러 메서드에 하드코딩	AbilityData(SO) + EffectData[]
발동 경로	보스/플레이어/몹 3갈래 제각각	전부 AbilityRunner.TryCast 단일 경로
새 공격 추가	새 클래스·메서드 작성 (프로그래머)	asset 조합 (디자이너 편집 가능)
스킬 단위	스킬 1개 = 클래스 1개 (상속)	스킬 = Effect 블록 조합
발수·간격	코드 상수(SerializeField)	데이터 값 (_count / _interval)
관심사	계산·연출·데미지·발동이 뒤엉킴	계산=Stat / 실행=Effect / 배달=투사체 / 발동=Runner

문제상황

전투 로직 3갈래 분산 · 하드코딩

“누가 쓰는 같은 파이프라인으로 흐르고, 새 공격은 데이터로 만든다”가 목표였다.

전투 로직이 세 갈래로 흩어져 있었다

- **보스** : BossSkillController(문자열 캐스트) + BossSkillBase 파생 클래스마다 1스킬. 신규 경로와 이중 공존.
- **플레이어** : BaseCharacter/유닛에 FireOneBullet·PerformAttackAction 하드코딩. 평타·Q·E 경로 제각각.
- **일반몹** : MonsterARController.RapidFireAsync 등 컨트롤러마다 발사 로직 직접 박힘.

구조적 문제 5가지

- 1 중복·비일관**
“공격을 발동한다”는 같은 개념이 캐스터 수만큼 따로 구현 → 유지보수 3배.
- 2 확장성 부재**
새 공격 = 코드를 새로 작성. 디자이너가 손댈 수 없음.
- 3 스킬 = 클래스 1개**
조합 불가. “데미지+상태이상+이펙트”를 붙이려면 매번 새 클래스.
- 4 관심사 미분리**
데미지 계산·연출·발동 조건·투사체 생성이 한 메서드에 뒤엉킴.
- 5 값이 코드에 박힘**
발수·간격·배율이 상수 → 변형마다 컴포넌트가 갈라짐.

통합 AbilitySystem — 단일 데이터 주도 파이프라인

캐스터 계약 → 러너 → 데이터(SO) → 실행 부품(IEffect). 전 진영이 같은 흐름을 탄다.

발동 파이프라인

캐스터	BaseCharacter · BaseMonsterController · BossMonsterController (IAbilityCaster)
발동	애니메이션 이벤트 / 입력 / Timeline Signal → runner.TryCast(ability, ctx, token)
러너	AbilityRunner — 캐스터마다 1개. 쿨다운·캐스트타임 관리 → BuildRuntimeEffects()
데이터	AbilityData(SO) = 코스트·쿨다운·캐스트타임 + EffectData[]
실행	IEffect.ExecuteAsync(ctx, token) 순서대로 — DealDamage · SpawnProjectiles · AreaStrike ...
적용	DamagePipeline(TakeDamage) · StatusSystem · 투사체 스냅샷 배달

핵심 타입

IAbilityCaster	캐스터 신원 표식(마커). 발동 진입점은 구현체마다
AbilityRunner	캐스터별 1개. 쿨다운·캐스트타임, TryCast
AbilityData(SO)	코스트·쿨다운·캐스트타임 + EffectData[]
EffectData→IEffect	편집용 SO → CreateRuntime() → 런타임 실행 부품

구체 Effect : DealDamage · SpawnProjectiles · SpreadProjectiles · AreaStrike · SpawnVFX · SummonMonsters

패턴·흐름 : RepeatEffect · ScatterPattern · DelayEffect · SelectRandomChoice · BranchByChoice

문제해결 (2)

스킬 = Effect 조합 + 슬롯 키 · 애니메이션 이벤트 통합

“좌클릭·Q·E마다 콜백을 따로” 대신, 상태가 슬롯을·intParameter가 비트 인덱스를 정하는 단일 콜백으로.

스킬 = 상속이 아니라 “Effect 조합”

라이플 5연사 = RepeatEffect(5, 0.05){ SpawnProjectiles(1), SpawnVFX(Muzzle) }

보스 광역기 = DealDamage → SpawnVFX → AreaStrike(경고→딜레이→폭발)

랜덤 분기 = SelectRandomChoice → BranchByChoice{ A:소환, B:포격 }

데미지·이펙트·상태이상을 붙이려면 클래스를 짜는 게 아니라 **asset에 Effect를 엮는다**.
발수·간격은 값 차이일 뿐.

AbilityBeat — 2단 조회 (상태 → 슬롯, intParameter → 비트)

slot = SlotForState(현재 상태) # 축1: 어느 스킬

if ((int)slot < 0) return; # 매칭 상태 아니면 무시

UseAbility(slot, e.intParameter); # 축2: 몇 번째 발사(비트 인덱스)

슬롯은 **어빌리티 리스트**(Dictionary<slot, List<AbilityData>>)를 갖는다. 인스펙터에 같은 슬롯을 순서대로 넣으면 [0][1][2] = 비트 인덱스.

효과

• **다단 발사**(모션 중 프레임마다 다른 투사체)를 코드 없이 데이터로 표현 — Q 클립 발사 프레임에 AbilityBeat(Int=비트)만 배치.

• **평타는 재배선 불필요**(항상 Int=0). E는 역할(Role)이 부여한 어빌리티가 Ex 슬롯 [0]으로 흡수돼 별도 경로 소멸.

문제해결 (3)

보스 단일화 · 왜 상속이 아니라 조합인가 · 결과

Timeline=연출/타이밍, AbilitySystem=발동. 세 진영이 하나의 계약으로 정렬된다.

보스 이중 시스템 → 단일화

이전 : Timeline Signal → BossSkillController.CastSkill("이름") → 구형 스킬 프리팹. 신규 CastAbility 경로와 이중 공존.

현재 : Timeline Signal → CastAbility(AbilityData) → AbilityRunner.TryCast 단일 경로. Timeline은 연출·타이밍만.

유물 선택도 편집 : BossRelicController가 IChoiceHandler를 구현 → SelectRandomChoice/BranchByChoice가 빨강/초록 분기 처리.

구형 BossSkillBase 프레임워크 8종 + 미사용 프리팹 전량 제거.

왜 "상속"이 아니라 "조합"인가

비교	상속 기반 (이전)	조합 기반 (현재)
스킬 단위	스킬 1개 = 클래스 1개 (조합 불가)	Effect 블록 조합 (자유 결합)
새 공격	새 코드 + 재컴파일	asset 편집 (디자이너 가능)
발동 경로	캐스터마다 제각각	TryCast 단일 경로
변형	값이 상수 → 클래스 분화	asset 복제로 변형

결과

- 3진영(플레이어·일반몹·보스) 전투 발동이 **하나의 데이터 주도 파이프라인**으로 수렴.
- 새 공격/스킬을 코드 없이 Effect 조합으로 제작 — 프로그래머 병목 제거.

- 다단 발사·랜덤 분기·소환·상태이상까지 부품 조합으로 표현.
- 보스 이중 시스템 제거 + 플레이어 팽타/E 이관 완료(Q는 인프라 완성).